



**COLLEGE OF ENGINEERING
THIRUVANANTHAPURAM**

**DEPARTMENT OF
ELECTRICAL ENGINEERING**

Laboratory Manual for

**OBJECT ORIENTED
PROGRAMMING LAB**

(PCEOL408)

Fourth Semester

B.Tech Electrical and Computer Engineering

(2024 Scheme - KTU)

February 2025

Preface

This is the first edition of the **OBJECT ORIENTED PROGRAMMING (JAVA) LAB** manual released by the Department of Electrical Engineering, College of Engineering, Thiruvananthapuram.

This material is designed to cater to the immediate needs of the students to meet the curriculum requirements of the **Fourth Semester B.Tech Electrical and Computer Engineering** students (2024 Scheme). A manual covering all programs in this book is being prepared for the faculty for verification and reference.

Every effort has been taken for the correctness of the subject dealt with; however, suggestions and remarks are welcome for future improvements.

Chief Co-ordinator

Head of the Department

(Faculty Name)

(Dr. Arun Kishore W C)

Prepared and compiled by:

1. (Faculty Name 1)
2. (Faculty Name 2)

DEPARTMENT VISION AND MISSION

VISION

“Be a centre of excellence and higher learning in Electrical Engineering and allied areas.”

MISSION

- To impart quality education in Electrical Engineering and bring-up professionally competent engineers.
- To mould ethically sound and socially responsible Electrical Engineers with leadership qualities.
- To inculcate research attitude and foster creativity in students.

SYLLABUS

PCEOL408 OBJECT ORIENTED PROGRAMMING (JAVA) LAB

Teaching Hours/Week (L:T:P:R): 0:0:3:0

CIE Marks: 50

Exam Hours: 2 Hrs. 30 Min.

Credits: 2

ESE Marks: 50

Course Objectives

1. To introduce Object Oriented Concepts: constructors, inheritance, method overloading & over-riding, and polymorphism in Java.
2. To practice robust application programs in Java using exception handling and threads.
3. To design and deploy applications using Java.

List of Experiments

(A) Basic programs using data types, operators and control statements:

1. Print any string to console.
2. Find average of three numbers.
3. Find frequency of a character in a string.
4. Multiply two given matrices.

(B) Utility programs in Java:

5. Menu driven program: Vector operations (create, insert, delete).
6. Read/Display integers from a line using StringTokenizer.
7. Random element selection from array (Random class).
8. Display calendar for specified month/year.

(C) Object Oriented Programming Concepts:

9. Student class with methods (insert, display).
10. Student class with Constructor.
11. Method Overloading (Area of Circle, Rectangle, Triangle).
12. Inheritance (Employee, Officer, Manager).
13. Method Overriding (Employee, Engineer).

(D) Exception handling and multi-threading:

14. Exception handling (try, catch, throws, finally).
15. Thread Synchronization.
16. Multi-threading (Two threads printing messages).

(E) Graphics Programming and JDBC:

17. Simple Calculator using Swing.
18. Factorial Calculator using Swing.
19. Display table records using JDBC.

Course Assessment Method (CIE: 50, ESE: 50)

Continuous Internal Evaluation (50 Marks)

- Attendance: 5
- Preparation/Pre-Lab: 25
- Internal Exam: 20

End Semester Examination (50 Marks)

- Procedure/Design/Algorithm: 10
- Conduct of Experiment/Execution: 15
- Result/Output: 10
- Viva Voce: 10
- Record: 5

Course Outcomes (COs)

- **CO1:** Implement OOP concepts (constructors, inheritance, polymorphism) in Java. (K3)
- **CO2:** Implement robust programs using exception handling. (K3)
- **CO3:** Implement multithreading and database connectivity. (K3)
- **CO4:** Implement GUI based applications using Swing. (K3)

Contents

1	Week 1: General Programs	6
2	Week 2: Strings and String Manipulation	38
3	Week 3: Object Oriented Programming	49
4	Week 4: Exception Handling and Multithreading	60
5	Week 5: File Handling & Collections	71
6	Week 6: GUI Programming (AWT/Swing) & Event Handling	82

Chapter 1

Week 1: General Programs

Important Note for Autograding

All programs in this week must follow strict Input/Output formats to pass the GitHub Classroom autograder.

- **No Prompts:** Do NOT print messages like "Enter a number:" or "The answer is:".
- **Exact Output:** Only print the specific output required by the problem statement.
- **Class Names:** Use the specific class name provided (or `Main` if unspecified).
- **Whitespace:** Ensure your output matches the example output exactly.

List of Experiments in Week 1

Experiment 1: Hello World	8
Experiment 2: Pattern Printing (Stars)	9
Experiment 3: Right-Angled Triangle	10
Experiment 4: HCF of Two Numbers	11
Experiment 5: Reverse Integer	12
Experiment 6: Sum of Natural Numbers	13
Experiment 7: Check Prime Number	14
Experiment 8: Factorial	15
Experiment 9: Trigonometric Functions	16
Experiment 10: Peak Element	17
Experiment 11: Largest Number at Least Twice of Others	18
Experiment 12: Second Largest Element	19
Experiment 13: Divisible Pairs	20
Experiment 14: First Occurrence	21
Experiment 15: Subarrays Divisible by K	22
Experiment 16: Sum of Array Except Self	23
Experiment 17: Sunrise Problem	24
Experiment 18: Cupcake Calorie Problem	25
Experiment 19: Matrix Transpose	26
Experiment 20: Boolean Matrix	27
Experiment 21: Chessboard Sum	28

Experiment 22: Alternate Matrix Traversal	29
Experiment 23: Rotate Image	30
Experiment 24: Spiral Matrix	31
Experiment 25: Bubble Sort Stats	32
Experiment 26: Median	33
Experiment 27: Insertion Sort Steps	34
Experiment 28: Super Digit	35
Experiment 29: Fibonacci Series	36
Experiment 30: Missing Number in Array	37

Experiment 1: Hello World

Problem Statement:

Welcome to the world of Programming! In this challenge, you will learn how to print “Hello World!” in standard output (stdout).

Output Format:

You must print two lines of output:

- Print “Hello, World.” on the first line (without quotes).
- Print “Hello, Java.” on the second line (without quotes).

Sample Output

```
Hello, World.  
Hello, Java.
```

Explanation

The program prints two fixed strings. The first line is "Hello, World." and the second line is "Hello, Java." as per the requirement.

Experiment 2: Pattern Printing (Stars)

Problem Statement:

In this task, you have to print n stars (*) vertically and n horizontally.

Input Format:

The first line of input contains n .

Output Format:

Print the stars in horizontal and vertical direction.

Example 1

Input:

3

Output:

* * *

*

*

*

Explanation

The output consists of a horizontal line of n stars separated by spaces, followed by n lines where each line contains a single star. Note that the first star of the vertical line essentially starts a new line after the horizontal row.

Experiment 3: Right-Angled Triangle

Problem Statement:

In this task, you need to print a right-angled triangle of height 3 (lines) and base 5 (stars).

Input Format:

No input.

Output Format:

Print the triangle of height 3 and base 5 as shown.

Example 1**Input:**

No input

Output:

*

* * *

* * * * *

Explanation

The shape printed is a right-angled triangle. Row 1: 1 star. Row 2: 3 stars. Row 3: 5 stars. This creates a triangle with a height of 3 lines and the last line (base) having 5 stars.

Experiment 4: HCF of Two Numbers

Problem Statement:

Given two numbers a and b , find their HCF. HCF or Highest Common Factor is the greatest common divisor between two numbers.

Input Format:

The first line of input contains the integers a and b .

Output Format:

The output should be the HCF of a and b .

Example 1**Input:**

75 90

Output:

15

Explanation:

$75 = 3 \times 5 \times 5$ and $90 = 2 \times 3 \times 3 \times 5$. So the common factors are $3 \times 5 = 15$.

Example 2**Input:**

25 50

Output:

25

Experiment 5: Reverse Integer

Problem Statement:

Write a program that prompts the user to input an integer and then outputs the number with the digits reversed. For example, if the input is 12345, the output should be 54321. Note: Input number will not have any trailing zeros.

Input Format:

The first line of input contains the integer n .

Output Format:

The output should be the reverse of n .

Example 1**Input:**

12345

Output:

54321

Explanation

We extract the last digit of the number using modulo operator (%), add it to the reversed number (after multiplying the current reversed number by 10), and then divide the original number by 10. Repeat until the number becomes 0.

Experiment 6: Sum of Natural Numbers

Problem Statement:

For an integer N , your task is to calculate the sum of first N natural numbers. Input User Task: You have to create a function `NumberSum()` which takes the integer (large value possible) N as a parameter and returns the sum of first N natural numbers.

Input Format:

First line is an integer N .

Output Format:

Print the sum of the first N natural numbers.

Example 1**Input:**

5

Output:

15

Explanation

The sum of the first 5 natural numbers is $1 + 2 + 3 + 4 + 5 = 15$.

Experiment 7: Check Prime Number

Problem Statement:

Write a program to check whether a given number is Prime or not. A prime number is a number greater than 1 that has no positive divisors other than 1 and itself.

Input Format:

A single integer n .

Output Format:

Print "Prime" if the number is prime, otherwise print "Not Prime".

Example 1**Input:**

7

Output:

Prime

Example 2**Input:**

10

Output:

Not Prime

Explanation

7 is only divisible by 1 and 7, so it is Prime. 10 is divisible by 1, 2, 5, 10, so it is Not Prime.

Experiment 8: Factorial

Problem Statement:

Write a program to find the factorial of a number. Create a function `factorial()` to do the operation.

Description:

The factorial of a non-negative integer n , denoted by $n!$, is the product of all positive integers less than or equal to n .

Example 1

Input:

5

Output:

120

Explanation

$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$.

Experiment 9: Trigonometric Functions

Problem Statement:

Write a program to find the sine and cosine of a given angle in degrees. Create two methods `sine()` and `cosine()`.

Note:

Java's `Math` library typically expects angles in radians. You may need to convert degrees to radians first.

Example 1

Input:

0

Output:

Sine: 0.0

Cosine: 1.0

Explanation

The sine of 0 degrees is 0. The cosine of 0 degrees is 1.

Experiment 10: Peak Element

Problem Statement:

An element is called a peak element if its value is not smaller than the value of its adjacent elements (if they exist). For first and last element consider only the element adjacent to it. Given an array `arr` of size n , find the index of first peak element. If peak element does not exist, print -1.

Input Format:

First line contains an integer n . Second line contains n space separated integers.

Output Format:

Return the index of the peak element in zero based indexing if present, else return -1.

Example 1**Input:**

4

5 10 20 15

Output:

2

Explanation:

20 is greater than both of its neighbours. Hence 20 is the peak element, so output is 2.

Example 2**Input:**

7

10 20 15 2 23 64 67

Output:

1

Experiment 11: Largest Number at Least Twice of Others

Problem Statement:

You are given an integer array `nums` of size n where the largest integer is unique. Your task is to determine whether the largest element in the array is at least twice as much as every other number in the array. If it is, print the index of the largest element, or print -1 otherwise.

Input Format:

The first line of the input contains the number n (length of the array). The next line contains n space separated integers denoting the elements of the array.

Output Format:

Print the index of the element if it satisfies the condition else print -1.

Example 1

Input:

4

3 6 1 0

Output:

1

Explanation:

6 is the largest integer. For every other number x , 6 is more than twice as big as x . Index is 1.

Example 2

Input:

4

1 2 3 4

Output:

-1

Explanation:

4 isn't at least as big as twice the value of 3.

Experiment 12: Second Largest Element

Problem Statement:

Given an unsorted array of size N find the 2nd largest element from the array without sorting the array.

Input Format:

The first line contains a single integer N . The second line consists of N integers of the array.

Output Format:

Print the second largest number in the new line.

Example 1**Input:**

6

3 2 1 5 6 4

Output:

5

Example 2**Input:**

9

10 1 7 2 5 6 8 9 3

Output:

9

Experiment 13: Divisible Pairs

Problem Statement:

Given an array of integers, `arr`, and a positive integer k , determine the number of (i, j) pairs in the array where $i < j$ and `arr[i] + arr[j]` is divisible by k .

Input Format:

Input consists of two lines. The first line contains 2 space-separated integers, n (size of array) and k . The second line contains n space-separated integers, each a value of `arr[i]`.

Output Format:

Print the number of pairs divisible by k , where $i < j$.

Example 1

Input:

6 3

1 3 2 6 1 2

Output:

5

Explanation:

Valid pairs: (0,2), (0,5), (1,3), (2,4), (4,5).

Example 2

Input:

4 5

1 3 2 6

Output:

1

Experiment 14: First Occurrence

Problem Statement:

Given an array `Arr` of N elements and a integer K . Your task is to print the position of first occurrence of K in the given array. If the number is not found, print -1. Note: Position of first element is considered as 1.

Input Format:

The first line contains two integers N and K . The second line contains N spaced integers, elements of `Arr`.

Output Format:

Print the position of first occurrence of K . If K is not present in the array, print -1.

Example 1

Input:

5 16

9 7 2 16 4

Output:

4

Example 2

Input:

6 6

5 7 7 8 8 10

Output:

-1

Experiment 15: Subarrays Divisible by K

Problem Statement:

Given an integer array `nums` and an integer k , print the number of non-empty subarrays that have a sum divisible by k . A subarray is a contiguous part of an array.

Input Format:

Input consists of two lines. The first line contains two integers n (size of the array) and k . The second line contains n spaced integers.

Output Format:

Print count of subarrays divisible by k .

Example 1

Input:

6 5

4 5 0 -2 -3 1

Output:

7

Explanation:

Subarrays: [4, 5, 0, -2, -3, 1], [5], [5, 0], [5, 0, -2, -3], [0], [0, -2, -3], [-2, -3].

Example 2

Input:

4 2

4 5 0 -2

Output:

4

Experiment 16: Sum of Array Except Self

Problem Statement:

Given an array of n integers where $n > 1$, return an array output such that `output[i]` is equal to the sum of all the elements of `nums` except `nums[i]`. All integers in array are greater than 0.

Input Format:

First line consists of an integer n which is the number of elements in array. Next n lines correspond to n elements of array.

Output Format:

Return the resultant array.

Example 1**Input:**

4

4 3 2 10

Output:

15 16 17 9

Explanation:

$a[0] = 3+2+10 = 15$

$a[1] = 4+2+10 = 16$

$a[2] = 4+3+10 = 17$

$a[3] = 4+3+2 = 9$

Experiment 17: Sunrise Problem

Problem Statement:

Given an array h representing heights of buildings. You have to count the buildings which will see the sunrise (Assume: Sun rise on the side of array starting point).

Input Format:

Line 1: contains an integer n denoting size of array. Line 2: contains n spaced integers denoting elements of array.

Output Format:

Print a single integer denoting the number of buildings which will see the sunrise.

Example 1**Input:**

5

7 4 8 2 9

Output:

3

Explanation:

7 can watch, 8 can watch, 9 can watch. Total 3.

Example 2**Input:**

6

6 2 8 4 11 13

Output:

4

Experiment 18: Cupcake Calorie Problem

Problem Statement:

Marc loves cupcakes. Each cupcake has a calorie count, and Marc can walk a distance to burn those calories. On eating the j^{th} cupcake with c calories, he must walk at least $(2^j) \times c$ miles to maintain his weight. Given the individual calorie counts for each of the cupcakes, determine the minimum number of miles Marc must walk to maintain his weight. Note that he can eat the cupcakes in any order.

Input Format:

The first line contains an integer n , the number of cupcakes. The next line inputs the calories of n cupcakes.

Output Format:

Print the minimum miles necessary.

Example 1

Input:

3

1 3 2

Output:

11

Explanation:

Eat 3 (calories) first: $0 + (3 \times 2^0) = 3$.

Eat 2 next: $3 + (2 \times 2^1) = 7$.

Eat 1 last: $7 + (1 \times 2^2) = 11$.

Experiment 19: Matrix Transpose

Problem Statement:

Write a program to find the transpose of a square matrix of size $N \times N$. Transpose of a matrix is obtained by changing rows to columns and columns to rows. Expected Time Complexity: $O(N \times N)$. Expected Space Complexity: Inplace.

Input Format:

The first line contains an integer N . The next N lines contains N spaced integers each, elements of matrix.

Output Format:

Print the transposed matrix.

Example 1**Input:**

```
4
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4
```

Output:

```
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
```

Explanation

The rows of the input matrix become the columns of the output matrix. Row 1 (1 1 1 1) becomes Column 1, Row 2 (2 2 2 2) becomes Column 2, and so on.

Experiment 20: Boolean Matrix

Problem Statement:

You are given a matrix `Mat` of m rows and n columns. The elements can only be either 0 or 1. If any row of the matrix contains a 1, then you need to fill that whole row with 1. After doing the mentioned operation, you need to return the modified matrix. Create function `BooleanMatrixProblem`.

Input Format:

The first line of contains m and n . Then next m lines contain n elements each.

Output Format:

Return the modified matrix.

Example 1

Input:

5 4

1 0 0 0

0 0 0 0

0 1 0 0

0 0 0 0

0 0 0 1

Output:

1 1 1 1

0 0 0 0

1 1 1 1

0 0 0 0

1 1 1 1

Experiment 21: Chessboard Sum

Problem Statement:

You are given a chessboard of size $N \times N$, where the top left square is black. Each square contains a value. Find the sum of the values of all black squares and all white squares. Remember that in a chessboard, black and white squares are alternate.

Input Format:

The first line contains N , the size of a row of the square matrix. The next N lines contain N space-separated integers each.

Output Format:

Print two lines, the first line containing the sum of black squares and the second line containing the sum of white squares.

Example 1

Input:

```
3
1 2 3
4 5 6
7 8 9
```

Output:

```
25
20
```

Experiment 22: Alternate Matrix Traversal

Problem Statement:

Given a 2D matrix of M rows and N columns, the task is to print the 2D matrix in an alternate manner. Elements of odd rows (1st, 3rd...) printed left to right. Elements of even rows (2nd, 4th...) printed right to left.

Input Format:

The first line contains two integers, M and N . The next M lines contain N space-separated integers.

Output Format:

One line with all elements after alternate traversal.

Example 1**Input:**

2 2

1 2

2 3

Output:

1 2 3 2

Experiment 23: Rotate Image

Problem Statement:

You are given a $n \times n$ 2-D matrix representing an image, rotate the image by 90 degrees (clockwise). Rotate the matrix inplace.

Input Format:

The first line will contain two integers n and n again. The next n lines contain n integers each representing the matrix.

Output Format:

Return a matrix that is rotated by 90 degrees in the clockwise direction.

Example 1**Input:**

2 2

1 2

2 3

Output:

2 1

3 2

Experiment 24: Spiral Matrix

Problem Statement:

Given a matrix of size $r \times c$, where r is number of rows and c is number of columns. Traverse the matrix in spiral form.

Input Format:

First line contains r and c . Next r lines contains c spaced integers.

Output Format:

Print the spiral matrix.

Example 1**Input:**

```
4 4
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 16
```

Output:

```
1 2 3 4 8 12 16 15 14 13 9 5 6 7 11 10
```


Experiment 25: Bubble Sort Stats

Problem Statement:

Perform Bubble Sort and print three values:

1. Number of swaps it took to sort.
2. First element after sorting.
3. Last element after sorting.

Input Format:

The first line contains an integer n . The second line contains n space-separated integers.

Output Format:

Print the required three values.

Example 1**Input:**

3

6 4 1

Output:

3

1

6

Explanation

Original: [6, 4, 1]. Pass 1: Swap 6,4 -> [4, 6, 1]. Swap 6,1 -> [4, 1, 6]. Swaps: 2. Pass 2: Swap 4,1 -> [1, 4, 6]. Swaps: 1. Total Swaps: 3. Sorted Array: 1, 4, 6. First: 1. Last: 6.

Experiment 26: Median

Problem Statement:

The median of a list of numbers is essentially its middle element after sorting. Given a list of numbers with an odd number of elements, find the median.

Input Format:

The first line inputs integer n , size of array. The second line inputs n space-separated integers.

Output Format:

In a new line, print the median of the array.

Example 1**Input:**

7

0 1 2 4 6 5 3

Output:

3

Experiment 27: Insertion Sort Steps

Problem Statement:

Given a sorted list with an unsorted number e in the rightmost cell, write code to insert e into the array so that it remains sorted. Print the array as a row of space-separated integers each time there is a shift or insertion.

Input Format:

The first line contains the integer n . The next line contains n space-separated integers.

Output Format:

Print the array at each shift or insertion.

Example 1**Input:**

5

2 4 6 8 3

Output:

2 4 6 8 8

2 4 6 6 8

2 4 4 6 8

2 3 4 6 8

Experiment 28: Super Digit

Problem Statement:

Given an integer, we need to find the super digit of the integer n which is concatenated k times.
Rules:

- If n has only 1 digit, then its super digit is n .
- Otherwise, the super digit of n is equal to the super digit of the sum of the digits of n .

Input Format:

The first line contains two space separated integers, n and k .

Output Format:

In a new line, print the the super digit of n repeated k times.

Example 1

Input:

148 3

Output:

3

Experiment 29: Fibonacci Series

Problem Statement:

Given an integer N , find the N^{th} number in the fibonacci series. Consider 0 and 1 to be the seed values. Series: 0, 1, 1, 2, 3, 5...

Input Format:

First line contains the integer N .

Output Format:

Print the N^{th} fibonacci number.

Example 1**Input:**

5

Output:

3

Explanation

The series starts 0, 1, 1, 2, 3... 1st: 0 2nd: 1 3rd: 1 4th: 2 5th: 3

Experiment 30: Missing Number in Array

Problem Statement:

Given an array containing n distinct numbers taken from $0, 1, 2, \dots, n$, find the one that is missing from the array.

Input Format:

First line contains integer n (size of the array range). Second line contains n space-separated integers.

Output Format:

Print the missing number.

Example 1**Input:**

3

3 0 1

Output:

2

Example 2**Input:**

2

0 1

Output:

2

Explanation

For Example 1: The array range is $[0, 3]$. The numbers present are 3, 0, 1. The missing number is 2. Logic: Sum of first n numbers is $n(n+1)/2$. Subtract the sum of array elements from this total to find the missing number.

Chapter 2

Week 2: Strings and String Manipulation

Important Note for Autograding

All programs in this week must follow strict Input/Output formats.

- Do NOT print prompts like "Enter string:".
- Only print the exact output required.
- Whitespace matters.

List of Experiments in Week 2

Experiment 31: Character Frequency	39
Experiment 32: Palindrome String	40
Experiment 33: Count Vowels, Consonants, Digits	41
Experiment 34: Reverse String Word by Word	42
Experiment 35: Anagram Check	43
Experiment 36: Largest Number from Strings	44
Experiment 37: Longest Common Prefix	45
Experiment 38: String Compression	46
Experiment 39: StringTokenizer (Sum of Integers)	47
Experiment 40: Sorting Strings	48

Experiment 31: Character Frequency

Problem Statement:

Write a Java program to find the frequency of a given character in a string.

Input Format:

First line: A string S .

Second line: A character C .

Output Format:

A single integer.

Example 1**Input:**

Hello World

l

Output:

3

Experiment 32: Palindrome String

Problem Statement:

Check whether a string is a palindrome.

Input Format:

A single string.

Output Format:

Print "1" if palindrome, "0" otherwise.

Example 1**Input:**

Malayalam

Output:

1

Experiment 33: Count Vowels, Consonants, Digits

Problem Statement:

Count vowels, consonants, digits, and special characters.

Input Format:

A single string.

Output Format:

Four space-separated integers: Vowels Consonants Digits Special.

Example 1**Input:**

JavaLab@2025

Output:

3 4 4 1

Experiment 34: Reverse String Word by Word

Problem Statement:

Reverse the string word by word.

Input Format:

A sentence string.

Output Format:

Reversed words string.

Example 1**Input:**

Hello World Java

Output:

Java World Hello

Experiment 35: Anagram Check

Problem Statement:

Check if two strings are anagrams.

Input Format:

Two strings on separate lines.

Output Format:

"true" or "false".

Example 1**Input:**

Triangle

Integral

Output:

true

Experiment 36: Largest Number from Strings

Problem Statement:

Form the largest number by appending strings.

Input Format:

First line: N .

Second line: N strings.

Output Format:

The largest number string.

Example 1**Input:**

3

10 3 31

Output:

33110

Experiment 37: Longest Common Prefix

Problem Statement:

Find the longest common prefix. Print "-1" if none.

Input Format:

First line: N .

Next N lines: Strings.

Output Format:

Prefix or "-1".

Example 1**Input:**

3

flower

flow

flight

Output:

fl

Experiment 38: String Compression

Problem Statement:

Compress string using counts (e.g., aab -> a2b1). Return original if compressed is longer.

Input Format:

A string.

Output Format:

Result string.

Example 1**Input:**

aabcccccaa

Output:

a2b1c5a3

Experiment 39: StringTokenizer (Sum of Integers)

Problem Statement:

Read a line of integers using StringTokenizer and display sum.

Input Format:

Line of integers.

Output Format:

Sum.

Example 1**Input:**

10 20 30

Output:

60

Experiment 40: Sorting Strings

Problem Statement:

Sort N names alphabetically.

Input Format:

First line: N .

Next N lines: Names.

Output Format:

Sorted names, one per line.

Example 1**Input:**

3

John

Alice

Bob

Output:

Alice

Bob

John

Chapter 3

Week 3: Object Oriented Programming

Important Note for Autograding

All programs in this week must follow strict Input/Output formats to pass the GitHub Classroom autograder.

- **No Prompts:** Do NOT print messages like "Enter name:".
- **Exact Output:** Only print the specific output required.
- **Class Structure:** Ensure strict adherence to the class structures defined in the problem statements.

List of Experiments in Week 3

Experiment 41: Area Calculation (Method Overloading)	50
Experiment 42: Student Class (Constructors)	51
Experiment 43: Inheritance (Employee Hierarchy)	52
Experiment 44: Complex Number Addition	53
Experiment 45: Method Overriding (Employee/Engineer)	54
Experiment 46: Abstract Class (Shapes)	55
Experiment 47: Interface (Resume)	56
Experiment 48: Garbage Collection	57
Experiment 49: 'this' Keyword	58
Experiment 50: 'super' Keyword (Constructor)	59

Experiment 41: Area Calculation (Method Overloading)

Problem Statement:

Write a Java program to calculate the area of different shapes using **Method Overloading**. Create a class 'AreaCalculator' with three overloaded methods named 'calculateArea': 1. 'calculateArea(float radius)': Calculates area of a circle (πr^2). 2. 'calculateArea(float length, float breadth)': Calculates area of a rectangle ($l \times b$). 3. 'calculateArea(double base, double height)': Calculates area of a triangle ($0.5 \times b \times h$).

Input Format:

First line: Radius of circle (float).

Second line: Length and Breadth of rectangle (float, space-separated).

Third line: Base and Height of triangle (double, space-separated).

Output Format:

Area of Circle (printed to 2 decimal places).

Area of Rectangle (printed to 2 decimal places).

Area of Triangle (printed to 2 decimal places).

Example 1

Input:

5

4 5

3 6

Output:

78.54

20.00

9.00

Experiment 42: Student Class (Constructors)

Problem Statement:

Create a 'Student' class with attributes 'name' (String) and 'rollNo' (int). Include a parameterized constructor to initialize these attributes. Write a main method to create two 'Student' objects with data read from input and display them.

Input Format:

Line 1: Name of student 1.

Line 2: Roll number of student 1.

Line 3: Name of student 2.

Line 4: Roll number of student 2.

Output Format:

Student 1 details (Name RollNo).

Student 2 details (Name RollNo).

Example 1

Input:

Alice

101

Bob

102

Output:

Name: Alice Roll: 101

Name: Bob Roll: 102

Experiment 43: Inheritance (Employee Hierarchy)

Problem Statement:

Create a class 'Employee' with attributes 'name', 'age', 'phoneNumber', 'address', and 'salary'. It should have a method 'printSalary()' to print the salary. Create two classes 'Officer' and 'Manager' that inherit from 'Employee'. 'Officer' has an additional attribute 'specialization'. 'Manager' has an additional attribute 'department'. Write a program to assign values to an Officer and a Manager and print their details.

Input Format:

Line 1-6 (Officer): Name, Age, Phone, Address, Salary, Specialization.

Line 7-12 (Manager): Name, Age, Phone, Address, Salary, Department.

Output Format:

Officer Details: Name, Age, Phone, Address, Salary, Specialization (one per line).

Manager Details: Name, Age, Phone, Address, Salary, Department (one per line).

Example 1

Input:

John
30
9876543210
Kerala
50000
IT
Jane
40
1234567890
India
80000
Sales

Output:

Officer:

John
30
9876543210
Kerala
50000
IT

Manager:

Jane
40
1234567890
India
80000
Sales

Experiment 44: Complex Number Addition

Problem Statement:

Create a class 'Complex' to represent a complex number with real and imaginary parts. Use a constructor to initialize the values. Include a method to add two complex numbers and return the result.

Input Format:

Line 1: Real and Imaginary parts of first number (space-separated).

Line 2: Real and Imaginary parts of second number (space-separated).

Output Format:

The sum in 'a + ib' format.

Example 1**Input:**

2 3

4 5

Output:

6 + 8i

Experiment 45: Method Overriding (Employee/Engineer)

Problem Statement:

Create a class 'Employee' with a method 'display()' that prints "Name of class is Employee". Create a subclass 'Engineer' that overrides 'display()' to print "Name of class is Engineer". In the main method, call 'display()' on an 'Engineer' object, then use the 'super' keyword to call the base class 'display()' method from within the 'Engineer' class.

Input Format:

No input.

Output Format:

Output of Engineer's display method.

Output of Employee's display method (called via super).

Example 1

Input:

No input

Output:

Name of class is Engineer

Name of class is Employee

Experiment 46: Abstract Class (Shapes)

Problem Statement:

Write a Java program to create an abstract class named 'Shape' that contains an empty method named 'numberOfSides()'. Provide three classes named 'Rectangle', 'Triangle', and 'Hexagon' such that each one of the classes extends the class 'Shape'. Each one of the classes contains only the method 'numberOfSides()' that shows the number of sides in the given geometrical structures.

Input Format:

No input.

Output Format:

Number of sides of Rectangle.

Number of sides of Triangle.

Number of sides of Hexagon.

Example 1

Input:

No input

Output:

Rectangle has 4 sides

Triangle has 3 sides

Hexagon has 6 sides

Experiment 47: Interface (Resume)

Problem Statement:

Create an interface 'Resume' with a method 'biodata()'. Implement this interface in a class 'Teacher' to display the resume details.

Input Format:

Line 1: Name.

Line 2: Qualification.

Line 3: Experience (years).

Output Format:

Name: [Name]

Qualification: [Qualification]

Experience: [Experience] years

Example 1**Input:**

Mary

MTech

5

Output:

Name: Mary

Qualification: MTech

Experience: 5 years

Experiment 48: Garbage Collection

Problem Statement:

Write a program to demonstrate garbage collection in Java. Create a class with a 'finalize()' method that prints "Garbage Collected". In the main method, create an object, assume it is null, and call the garbage collector.

Input Format:

No input.

Output Format:

Garbage Collected

Example 1**Input:**

No input

Output:

Garbage Collected

Note: Output may vary depending on JVM behavior, but logic must be correct.

Experiment 49: 'this' Keyword

Problem Statement:

Write a program to demonstrate the use of the 'this' keyword to distinguish between class attributes and method parameters with the same name.

Input Format:

An integer value.

Output Format:

The value set in the instance variable.

Example 1**Input:**

50

Output:

Value: 50

Experiment 50: 'super' Keyword (Constructor)

Problem Statement:

Write a program to demonstrate the use of 'super' keyword to invoke the constructor of the parent class. Create a class 'Person' with a constructor printing "Person class". Create a subclass 'Student' with a constructor printing "Student class" and calling 'super()'.

Input Format:

No input.

Output Format:

Person class

Student class

Example 1**Input:**

No input

Output:

Person class

Student class

Chapter 4

Week 4: Exception Handling and Multi-threading

Important Note for Autograding

For Exception Handling experiments, ensure the error messages match the output format exactly. For Multithreading, strict output ordering may vary due to concurrency, but the logic must be correct.

List of Experiments in Week 4

Experiment 51: Division with Exception Handling	61
Experiment 52: Multiple Catch Blocks (Array/Arithmetic)	62
Experiment 53: Custom Exception (Age Validation)	63
Experiment 54: Finally Block	64
Experiment 55: Throws Keyword	65
Experiment 56: Multithreading (Extending Thread Class)	66
Experiment 57: Multithreading (Runnable Interface)	67
Experiment 58: Thread Synchronization	68
Experiment 59: Producer Consumer Problem	69
Experiment 60: Thread Priorities	70

Experiment 51: Division with Exception Handling

Problem Statement:

Write a Java program that reads two integers a and b and calculates a/b . Use a try-catch block to handle `ArithmeticException` if b is zero. If an exception occurs, print "Divide by zero error". Otherwise, print the result.

Input Format:

Two integers a and b .

Output Format:

The result of division OR "Divide by zero error".

Example 1**Input:**

10 2

Output:

5

Example 2**Input:**

10 0

Output:

Divide by zero error

Experiment 52: Multiple Catch Blocks (Array/Arithmetic)

Problem Statement:

Write a program that takes an array size N and N integers. Then it takes an index and a divisor. Perform division of the element at the specified index by the divisor. Handle `ArrayIndexOutOfBoundsException` (print "Invalid Index") and `ArithmeticException` (print "Divide by zero error") using multiple catch blocks.

Input Format:

Line 1: N .
Line 2: N integers.
Line 3: Index to access.
Line 4: Divisor.

Output Format:

Result or Error Message.

Example 1

Input:

3
10 20 30
1
0

Output:

Divide by zero error

Example 2

Input:

3
10 20 30
5
2

Output:

Invalid Index

Experiment 53: Custom Exception (Age Validation)

Problem Statement:

Create a custom exception class named `InvalidAgeException`. Write a program to read an age. If age is less than 18, throw the custom exception with the message "Not eligible to vote". Otherwise, print "Eligible to vote".

Input Format:

An integer representing age.

Output Format:

"Eligible to vote" OR "Exception: Not eligible to vote".

Example 1**Input:**

15

Output:

Exception: Not eligible to vote

Experiment 54: Finally Block

Problem Statement:

Write a program to demonstrate the use of the `finally` block. Perform a division operation inside a try block. Regardless of whether an exception occurs or not, the finally block should execute and print "Finally block executed".

Input Format:

Two integers a and b .

Output Format:

Result (or Exception message) followed by "Finally block executed".

Example 1

Input:

10 2

Output:

Result: 5

Finally block executed

Example 2

Input:

5 0

Output:

Error

Finally block executed

Experiment 55: Throws Keyword

Problem Statement:

Create a method `check(int n)` that declares it throws an `IOException` if $n < 0$. In the main method, call this method and handle the exception. If $n \geq 0$, print the number.

Input Format:

An integer n .

Output Format:

The number or "Caught: IO Exception".

Example 1**Input:**

-5

Output:

Caught: IO Exception

Experiment 56: Multithreading (Extending Thread Class)

Problem Statement:

Create a class `MyThread` that extends `Thread`. The thread should print "Thread is running". Write a main method to create and start the thread.

Input Format:

No input.

Output Format:

Thread is running

Example 1**Input:**

No input

Output:

Thread is running

Experiment 57: Multithreading (Runnable Interface)

Problem Statement:

Create a class `MyRunnable` that implements `Runnable`. The `run` method should print numbers from 1 to 5. Create a thread using this runnable and start it.

Input Format:

No input.

Output Format:

1 2 3 4 5 (space or newline separated).

Example 1**Input:**

No input

Output:

1 2 3 4 5

Experiment 58: Thread Synchronization

Problem Statement:

Write a program to print the multiplication table of a number using a synchronized method. Create two threads that attempt to print the table of 5 and 100 respectively.

Input Format:

No Input.

Output Format:

Multiplication table of 5 followed by table of 100 (or vice versa, but complete tables must be printed without interleaving).

Sample Output

```
5 10 15 20 25
100 200 300 400 500
```

(Note: Only first 5 multiples for brevity in sample).

Experiment 59: Producer Consumer Problem

Problem Statement:

Implement the Producer-Consumer problem using `wait()` and `notify()` methods for inter-thread communication. The producer should produce 5 items and the consumer should consume them.

Input Format:

No Input.

Output Format:

Produced: 1
Consumed: 1
... up to 5.

Sample Output

```
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
```

Experiment 60: Thread Priorities

Problem Statement:

Write a program to create three threads. Set the priority of the first to MIN_PRIORITY, second to NORM_PRIORITY, and third to MAX_PRIORITY. Display the name and priority of each thread.

Input Format:

No Input.

Output Format:

Thread 1 Priority: 1

Thread 2 Priority: 5

Thread 3 Priority: 10

Example 1**Input:**

No input

Output:

Thread 1 Priority: 1

Thread 2 Priority: 5

Thread 3 Priority: 10

Chapter 5

Week 5: File Handling & Collections

Important Note for Autograding

File I/O: For file experiments, assume the input files (e.g., `input.txt`) are present in the working directory. Do NOT use absolute paths (e.g., `C:/Users/...`). Use relative paths.

List of Experiments in Week 5

Experiment 61: File Read with Line Numbers	72
Experiment 62: File Copy (Byte Streams)	73
Experiment 63: File Statistics	74
Experiment 64: Vector Operations	75
Experiment 65: Even/Odd File Separation	76
Experiment 66: ArrayList Operations	77
Experiment 67: LinkedList Traversal	78
Experiment 68: Frequency Dictionary (HashMap)	79
Experiment 69: Unique Elements (HashSet)	80
Experiment 70: Generics (Max Element)	81

Experiment 61: File Read with Line Numbers

Problem Statement:

Write a program to read a text file named `input.txt` and display its contents on the console. Each line should be preceded by its line number. (*Note: Assume `input.txt` exists*).

Input Format:

The program reads from `input.txt`. No console input.

Output Format:

Line 1: content...

Line 2: content...

Example (File Content)

Hello World
Java Programming

Example Output

1: Hello World
2: Java Programming

Experiment 62: File Copy (Byte Streams)

Problem Statement:

Write a program to copy the contents of `source.txt` to `destination.txt` using `FileInputStream` and `FileOutputStream`. After copying, print "File Copied" and display the content of the destination file.

Input Format:

No console input. Reads from `source.txt`.

Output Format:

File Copied

(Contents of `destination.txt`)

Example (source.txt content)

Copy this text.

Example Output

File Copied

Copy this text.

Experiment 63: File Statistics

Problem Statement:

Write a program that reads a file `data.txt` and counts the number of characters, words, and lines in it.

Input Format:

No console input. Reads from `data.txt`.

Output Format:

Characters: [count]

Words: [count]

Lines: [count]

Example (data.txt content)

```
Hello Java
Welcome
```

Example Output

```
Characters: 18
Words: 3
Lines: 2
```

Note: Character count logic may vary slightly based on newline handling (CRLF vs LF). Ensure logic counts visible characters and spaces.

Experiment 64: Vector Operations (Syllabus Section B)

Problem Statement:

Write a menu-driven Java program to manage a list of student names using the `Vector` class. Implement the following options: 1. Add a student name. 2. Insert a name at a specific index (1-based index input). 3. Remove a name. 4. Display the list. 5. Exit.

Input Format:

Integers for menu choice. Strings for names. Integers for index.

Output Format:

Specific messages for each operation as shown in example.

Example Interaction

Input:

```
1 Alice
1 Bob
2 Charlie 2
4
3 Bob
4
5
```

Output:

```
Added
Added
Inserted
[Alice, Charlie, Bob]
Removed
[Alice, Charlie]
```

Experiment 65: Even/Odd File Separation

Problem Statement:

Write a program that reads a file `numbers.txt` containing integers. Write all even numbers to `even.txt` and all odd numbers to `odd.txt`. Finally, display the contents of both output files.

Input Format:

No console input. Reads from `numbers.txt` (space or newline separated integers).

Output Format:

Even File: [content]

Odd File: [content]

Example (numbers.txt)

10 5 8 3 2

Example Output

Even File: 10 8 2

Odd File: 5 3

Experiment 66: ArrayList Operations

Problem Statement:

Create an ArrayList of Strings representing items in a shopping cart. Perform the following: 1. Add items. 2. Sort the items alphabetically. 3. Check if a specific item exists. 4. Print the final list.

Input Format:

Line 1: Number of items N .

Line 2: N Strings (items).

Line 3: Item to search.

Output Format:

Sorted Items: [List]

Found/Not Found

Example 1

Input:

3

Milk Bread Eggs

Bread

Output:

Sorted Items: [Bread, Eggs, Milk]

Found

Experiment 67: LinkedList Traversal

Problem Statement:

Create a `LinkedList` of integers. Use a `ListIterator` to traverse the list in forward direction and then in backward direction.

Input Format:

Line 1: N .

Line 2: N integers.

Output Format:

Forward: [elements]

Backward: [elements]

Example 1

Input:

3

10 20 30

Output:

Forward: 10 20 30

Backward: 30 20 10

Experiment 68: Frequency Dictionary (HashMap)

Problem Statement:

Read a line of text. Use a `HashMap` to count the frequency of every word in the text. Print the unique words and their counts.

Input Format:

A single line of text.

Output Format:

Word: Count (in any order).

Example 1**Input:**

hello world hello java

Output:

hello: 2

world: 1

java: 1

Experiment 69: Unique Elements (HashSet)

Problem Statement:

Read N integers from the user. Use a `HashSet` to remove duplicates and print only the unique numbers.

Input Format:

Line 1: N .

Line 2: N integers.

Output Format:

Unique numbers (space separated).

Example 1**Input:**

5

10 20 10 30 20

Output:

10 20 30

Note: Order in `HashSet` is not guaranteed, but for this experiment, printing elements is sufficient.

Experiment 70: Generics (Max Element)

Problem Statement:

Write a generic method `findMax(T[] array)` that takes an array of any type (that implements `Comparable`) and returns the maximum element. Demonstrate this with an Integer array and a String array.

Input Format:

Line 1: N (size of int array)

Line 2: N integers.

Line 3: M (size of string array)

Line 4: M strings.

Output Format:

Max Integer: [val]

Max String: [val]

Example 1

Input:

3

10 50 20

3

Apple Orange Banana

Output:

Max Integer: 50

Max String: Orange

Chapter 6

Week 6: GUI Programming (AWT/Swing) & Event Handling

Important Note for Evaluation

Since these are Graphical User Interface (GUI) programs, they produce visual output in windows rather than text on the console. Evaluation will be done by running the program and verifying the interactive behavior (e.g., clicking buttons, observing color changes).

List of Experiments in Week 6

Experiment 71: Simple Calculator (Swing)	83
Experiment 72: Traffic Light Simulator	84
Experiment 73: Mouse Listener	85
Experiment 74: Key Listener	86
Experiment 75: Adapter Classes (WindowAdapter)	87
Experiment 76: Applet Lifecycle	88
Experiment 77: Graphics (Drawing Shapes)	89
Experiment 78: Parameter Passing to Applet	90
Experiment 79: Mouse Motion (Freehand Drawing)	91
Experiment 80: Background Color Changer	92

Experiment 71: Simple Calculator (Swing)

Problem Statement:

Create a simple Calculator using Java Swing. The GUI should have three text fields: two for inputting numbers and one for displaying the result. Include four buttons labeled '+', '-', '*', and '/'. When a button is clicked, perform the corresponding arithmetic operation on the input numbers and display the result in the result field.

GUI Components:

- 3 JTextField (Input 1, Input 2, Result)
- 4 JButton (Add, Subtract, Multiply, Divide)
- Labels for each field.

Expected Behavior:

User enters 10 and 5. Clicks '+'. Result field shows 15.

Experiment 72: Traffic Light Simulator

Problem Statement:

Simulate a Traffic Light using 'JRadioButton'. Create a group of three radio buttons labeled "Red", "Yellow", and "Green". When a button is selected, display a message (e.g., "Stop", "Ready", "Go") or change the background color of a panel to match the selected light.

GUI Components:

- 3 JRadioButton in a ButtonGroup.
- A display area (Label or Panel) to show the status.

Expected Behavior:

Select "Red" → Display "Stop" or turn panel Red.

Experiment 73: Mouse Listener

Problem Statement:

Write a Java program that implements the 'MouseListener' interface. The program should display the current mouse event (Clicked, Entered, Exited, Pressed, Released) along with the coordinates (X, Y) on a Label or in the console.

GUI Components:

- A 'Frame' or 'JFrame'.
- A 'Label' to display status.

Expected Behavior:

Clicking anywhere in the window updates the label to "Mouse Clicked at [x, y]".

Experiment 74: Key Listener

Problem Statement:

Implement the 'KeyListener' interface. Create a GUI with a text area or frame that captures keyboard input. Display the key that was typed, pressed, or released in a label.

GUI Components:

- A 'Frame' or 'JFrame'.
- A 'Label' to show the key state.

Expected Behavior:

Pressing 'A' updates the label to "Key Typed: a".

Experiment 75: Adapter Classes (WindowAdapter)

Problem Statement:

Write a program to demonstrate the use of an Adapter Class. Use 'WindowAdapter' to handle the window closing event, so that the application terminates correctly when the close button (X) is clicked. This avoids implementing all methods of the 'WindowListener' interface.

GUI Components:

- A simple 'Frame'.

Expected Behavior:

The window closes and the program terminates when the 'X' button is clicked.

Experiment 76: Applet Lifecycle

Problem Statement:

Write a Java Applet that overrides 'init()', 'start()', 'stop()', and 'destroy()' methods. Add print statements in each method to display messages to the console (or draw string on the applet window) to demonstrate the order of execution.

Expected Behavior:

Console output showing:

"Applet Initialized"

"Applet Started"

(On minimize) "Applet Stopped"

(On close) "Applet Destroyed"

Experiment 77: Graphics (Drawing Shapes)

Problem Statement:

Write a program (Applet or Frame) that uses the 'Graphics' class to draw basic shapes. Draw a rectangle, an oval, a line, and a string text "Hello Graphics" in different colors.

GUI Components:

- A canvas/panel to draw on.

Expected Behavior:

A window displaying colorful geometric shapes.

Experiment 78: Parameter Passing to Applet

Problem Statement:

Write an Applet that accepts a parameter named "Message" from an HTML file (using the '<param>' tag) and displays that message in the Applet window.

Input (HTML):

```
<param name="Message" value="Welcome to Java Applets">
```

Expected Behavior:

The Applet window displays "Welcome to Java Applets".

Experiment 79: Mouse Motion (Freehand Drawing)

Problem Statement:

Implement the 'MouseListener' interface to create a simple freehand drawing tool. When the user drags the mouse, draw lines following the mouse path on the window.

GUI Components:

- A drawing area (Frame/Panel).

Expected Behavior:

Dragging the mouse draws a continuous line (like a pencil tool).

Experiment 80: Background Color Changer

Problem Statement:

Create a GUI with three buttons labeled "Red", "Green", and "Blue". When a button is clicked, the background color of the window should change to the corresponding color. Use 'ActionListener' to handle the events.

GUI Components:

- 3 JButton.
- Main 'JFrame' container.

Expected Behavior:

Clicking "Red" makes the window background red.